

Top 100 Must Know SQL Queries

1. CORE SELECT RETRIEVAL

1. Select all columns from a table

```
SELECT * FROM employees;
```

2. Select specific columns

```
SELECT first_name, last_name FROM employees;
```

3. Rename columns using alias

```
SELECT salary AS emp_salary FROM employees;
```

4. Filter rows using WHERE

```
SELECT * FROM orders WHERE status = 'Shipped';
```

5. Use IN to match multiple values

```
SELECT * FROM customers WHERE country IN ('India', 'USA', 'UK');
```

6. Use BETWEEN for range filters

```
SELECT * FROM products WHERE price BETWEEN 100 AND 500;
```

7. Pattern matching using LIKE

```
SELECT * FROM users WHERE email LIKE '%@gmail.com';
```

8. Check for NULL values

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

9. Get unique values using DISTINCT

```
SELECT DISTINCT department FROM employees;
```

10. Sort results using ORDER BY

```
SELECT * FROM orders ORDER BY order_date DESC;
```

2. AGGREGATION & GROUPING

11. Count rows grouped by column

```
SELECT department, COUNT(*) FROM employees GROUP BY department;
```

12. Aggregate multiple columns

```
SELECT department, COUNT(*), AVG(salary) FROM employees GROUP BY department;
```

13. Use HAVING to filter groups

```
SELECT department, COUNT(*) FROM employees GROUP BY department  
HAVING COUNT(*) > 5;
```

14. Group by multiple columns

```
SELECT department, role, COUNT(*) FROM employees GROUP BY  
department, role;
```

15. Calculate percentage of total

```
SELECT category, COUNT(*) * 100.0 / SUM(COUNT(*)) OVER () AS  
percentage FROM products GROUP BY category;
```

16. Filter with aggregate inside WHERE

```
SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM  
employees);
```

17. Use COUNT DISTINCT

```
SELECT COUNT(DISTINCT user_id) FROM logins;
```

18. Use conditional aggregation

```
SELECT COUNT(*) FILTER (WHERE status = 'Open') FROM tickets;
```

19. Group by date

```
SELECT DATE(created_at), COUNT(*) FROM users GROUP BY  
DATE(created_at);
```

20. Top N per group using ROW_NUMBER

(See window functions below.)

3. JOINS (INNER, OUTER, SELF)

21. INNER JOIN two tables

```
SELECT * FROM orders JOIN customers ON orders.customer_id =  
customers.id;
```

22. LEFT JOIN (keep all from left)

```
SELECT * FROM customers LEFT JOIN orders ON customers.id =  
orders.customer_id;
```

23. RIGHT JOIN

```
SELECT * FROM orders RIGHT JOIN customers ON orders.customer_id =  
customers.id;
```

24. FULL OUTER JOIN

```
SELECT * FROM a FULL OUTER JOIN b ON a.id = b.id;
```

25. CROSS JOIN (cartesian product)

```
SELECT * FROM sizes CROSS JOIN colors;
```

26. SELF JOIN (same table twice)

```
SELECT e1.name, e2.name FROM employees e1 JOIN employees e2 ON  
e1.manager_id = e2.id;
```

27. JOIN with multiple conditions

```
SELECT * FROM orders JOIN items ON orders.id = items.order_id AND  
items.status = 'Delivered';
```

28. Anti-join (not in another table)

```
SELECT * FROM users WHERE id NOT IN (SELECT user_id FROM logins);
```

29. Semi-join (exists in another)

```
SELECT * FROM users WHERE EXISTS (SELECT 1 FROM logins WHERE  
logins.user_id = users.id);
```

30. Join on date ranges

```
SELECT * FROM users JOIN plans ON users.joined_date BETWEEN  
plans.start_date AND plans.end_date;
```

4. SUBQUERIES & SET OPERATIONS

31. Subquery in SELECT

```
SELECT name, (SELECT COUNT(*) FROM orders WHERE user_id =  
users.id) AS order_count FROM users;
```

32. Subquery in WHERE

```
SELECT * FROM employees WHERE department_id = (SELECT id FROM  
departments WHERE name = 'Sales');
```

33. EXISTS and NOT EXISTS

```
SELECT * FROM users WHERE EXISTS (SELECT 1 FROM logins WHERE  
logins.user_id = users.id);
```

34. IN with subquery

```
SELECT * FROM users WHERE id IN (SELECT user_id FROM orders);
```

35. Correlated subquery

```
SELECT * FROM orders o WHERE amount > (SELECT AVG(amount) FROM  
orders WHERE user_id = o.user_id);
```

36. UNION ALL

```
SELECT name FROM employees UNION ALL SELECT name FROM  
contractors;
```

37. UNION (removes duplicates)

```
SELECT email FROM students UNION SELECT email FROM teachers;
```

38. INTERSECT

```
SELECT email FROM customers INTERSECT SELECT email FROM  
newsletter_subscribers;
```

39. EXCEPT / MINUS

```
SELECT email FROM users EXCEPT SELECT email FROM unsubscribed;
```

40. Subquery in FROM (derived table)

```
SELECT * FROM (SELECT COUNT(*) FROM orders) AS order_count;
```

5. WINDOW FUNCTIONS

41. ROW_NUMBER

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) FROM employees;
```

42. RANK / DENSE_RANK

```
SELECT *, RANK() OVER (ORDER BY score DESC) FROM tests;
```

43. LAG / LEAD

```
SELECT name, salary, LAG(salary) OVER (ORDER BY id) AS prev_salary FROM employees;
```

44. Cumulative SUM

```
SELECT date, SUM(amount) OVER (ORDER BY date) FROM sales;
```

45. Moving average

```
SELECT date, AVG(amount) OVER (ORDER BY date ROWS 6 PRECEDING) FROM sales;
```

46. FIRST_VALUE and LAST_VALUE

```
SELECT name, FIRST_VALUE(score) OVER (PARTITION BY subject ORDER BY date) FROM results;
```

47. NTILE (bucket values)

```
SELECT *, NTILE(4) OVER (ORDER BY salary) AS salary_quartile FROM employees;
```

48. Percent of total

```
SELECT department, COUNT(*) * 100.0 / SUM(COUNT(*)) OVER ( ) FROM employees GROUP BY department;
```

49. Conditional count with window

```
SELECT date, COUNT(*) FILTER (WHERE status = 'Open') OVER (ORDER BY date) FROM tickets;
```

50. Calculate gap between events

```
SELECT id, event_time, LAG(event_time) OVER (ORDER BY event_time) FROM logs;
```

6. COMMON TABLE EXPRESSIONS (CTE)

51. Basic CTE

```
WITH recent_orders AS (SELECT * FROM orders WHERE order_date >=
'2024-01-01') SELECT * FROM recent_orders;
```

52. Multiple CTEs

```
WITH step1 AS (...), step2 AS (...) SELECT * FROM step2;
```

53. Recursive CTE (e.g. org chart)

```
WITH RECURSIVE emp_tree AS (...) SELECT * FROM emp_tree;
```

54. Generate series using recursive CTE

```
WITH RECURSIVE nums AS (SELECT 1 AS n UNION ALL SELECT n + 1 FROM
nums WHERE n < 100) SELECT * FROM nums;
```

55. Chain CTE with window function

```
WITH ranked_orders AS (...) SELECT * FROM ranked_orders WHERE rn
= 1;
```

56. Use CTE for filtering aggregates

```
WITH counts AS (SELECT dept, COUNT(*) AS cnt FROM emp GROUP BY
dept) SELECT * FROM counts WHERE cnt > 10;
```

57. CTE as subquery for UPDATE

(Use with care per RDBMS)

58. Recursive hierarchy with level

```
WITH RECURSIVE emp(id, manager_id, lvl) AS (...) SELECT * FROM
emp;
```

59. Recursive path generation

```
WITH RECURSIVE paths AS (...) SELECT * FROM paths;
```

60. Build calendar table with CTE

```
WITH RECURSIVE dates AS (...) SELECT * FROM dates;
```

7. DATA MODIFICATION (DML)

61. INSERT new row

```
INSERT INTO products (name, price) VALUES ('Laptop', 50000);
```

62. INSERT from SELECT

```
INSERT INTO archive SELECT * FROM orders WHERE created_at <
'2024-01-01';
```

63. UPDATE specific rows

```
UPDATE users SET is_active = FALSE WHERE last_login <
'2024-01-01';
```

64. DELETE rows

```
DELETE FROM sessions WHERE expired = TRUE;
```

65. UPSERT (Postgres: ON CONFLICT)

```
INSERT INTO users (id, email) VALUES (...) ON CONFLICT (id) DO
UPDATE SET email = EXCLUDED.email;
```

66. MERGE (SQL Server / Snowflake)

```
MERGE INTO target USING source ON ... WHEN MATCHED THEN UPDATE
... WHEN NOT MATCHED THEN INSERT ...;
```

67. RETURNING clause (Postgres)

```
DELETE FROM users WHERE id = 10 RETURNING *;
```

68. Transactions

```
BEGIN; UPDATE...; DELETE...; COMMIT;
```

69. SAVEPOINT and ROLLBACK

```
SAVEPOINT my_save; ROLLBACK TO my_save;
```

70. DELETE with subquery

```
DELETE FROM users WHERE id IN (SELECT user_id FROM banned_users);
```

8. DDL & ADMIN TASKS

71. CREATE TABLE

```
CREATE TABLE students (id INT PRIMARY KEY, name TEXT, age INT);
```

72. ALTER TABLE

```
ALTER TABLE students ADD COLUMN email TEXT;
```

73. DROP TABLE

```
DROP TABLE IF EXISTS temp_data;
```

74. CREATE INDEX

```
CREATE INDEX idx_name ON users(name);
```

75. GRANT / REVOKE

```
GRANT SELECT ON orders TO analyst;
```

9. PERFORMANCE & DEBUGGING

76. EXPLAIN query plan

```
EXPLAIN ANALYZE SELECT * FROM orders WHERE id = 1;
```

77. CHECK index usage

```
SELECT * FROM pg_stat_user_indexes;
```

78. REFRESH MATERIALIZED VIEW

```
REFRESH MATERIALIZED VIEW sales_summary;
```

79. VACUUM / ANALYZE

```
VACUUM ANALYZE tablename;
```

80. Find slow queries

```
SELECT * FROM pg_stat_activity WHERE state = 'active';
```

10. JSON, ARRAYS & ADVANCED

81. Extract from JSON

```
SELECT data->>'name' FROM users;
```

82. Aggregate JSON objects

```
SELECT json_agg(row_to_json(t)) FROM (SELECT * FROM users) t;
```

83. UNNEST arrays

```
SELECT unnest(array_column) FROM table;
```

84. Create array from rows

```
SELECT array_agg(id) FROM users;
```

85. Filter inside JSON

```
SELECT * FROM logs WHERE data->>'event' = 'login';
```

11. MISCELLANEOUS TRICKS

86. Generate UUID

```
SELECT gen_random_uuid();
```

87. Convert string to date

```
SELECT TO_DATE('2025-01-01', 'YYYY-MM-DD');
```

88. Case statements

```
SELECT name, CASE WHEN age < 18 THEN 'Minor' ELSE 'Adult' END  
FROM users;
```

89. REGEXP functions

```
SELECT * FROM users WHERE name ~ '^[A-Z]';
```

90. Pivot data (SQL Server example)

Use **PIVOT** function or conditional aggregation.

12. MONITORING & METADATA

91. List tables

```
SELECT table_name FROM information_schema.tables WHERE  
table_schema = 'public';
```

92. Describe table structure

```
DESCRIBE table_name;
```

93. Count rows in each table

```
SELECT table_name, COUNT(*) FROM table_name GROUP BY table_name;
```

94. Find locked queries

```
SELECT * FROM pg_locks WHERE granted = FALSE;
```

95. Session variables

```
SELECT current_user, current_schema;
```

13. DATE & TIME LOGIC

96. Current timestamp

```
SELECT CURRENT_TIMESTAMP;
```

97. Extract parts of date

```
SELECT EXTRACT(YEAR FROM order_date) FROM orders;
```

98. Date difference

```
SELECT end_date - start_date FROM bookings;
```

99. Add interval

```
SELECT order_date + INTERVAL '7 days' FROM orders;
```

100. Format date

```
SELECT TO_CHAR(order_date, 'YYYY-MM-DD') FROM orders;
```